Введение в Prisma ORM

Подключение Prisma 7 к готовой PostgreSQL-базе: модели, связи и первое чтение

Результаты занятия

- Объяснять назначение ORM и Prisma Client в приложении Node.js.
- Различать реальную таблицу PostgreSQL и модель Prisma.
- Установить Prisma ORM 7 и настроить доступ к shop_db.
- Получить модели существующей базы командой prisma db pull.
- Согласовать модели Product и Category со столбцами и внешним ключом.
- Сгенерировать Prisma Client и выполнить один тестовый запрос на чтение.

Учебные вопросы

№	Вопрос	Основной результат
1	Понятие ORM	Роль моделей при работе с SQL-базой
2	Устройство Prisma ORM	Schema, Client, PostgreSQL и db pull
3	Модели и отношения	Product, Category, @map, @@map, @relation
4	Подключение Prisma 7	Пакеты, конфигурация, генерация клиента
5	Проверка соединения	Один запрос findMany() без изменения REST API

ORM и модели данных

Знакомимся с ORM без подробного CRUD

Что такое ORM

- ORM — подход для работы с реляционной БД через модели и методы языка программирования.
- PostgreSQL остаётся местом хранения данных.
- SQL по-прежнему выполняется СУБД, но обычные запросы формируются через ORM.
- Node.js вызывает методы Prisma Client вместо ручной сборки части SQL.

Ручной SQL и ORM

node-postgres + SQL

- Разработчик пишет SELECT и другие SQL-команды.
- Использует pool.query() и параметры $1, $2.
- Получает result.rows.
- Может полностью контролировать текст запроса.

Prisma ORM

- Разработчик вызывает методы Prisma Client.
- Фильтры задаются JavaScript-объектами.
- Модели описывают поля и связи.
- Результатом становятся JavaScript-объекты.

Одна операция двумя способами

```javascript
// node-postgres
const result = await pool.query(
  "SELECT * FROM products WHERE id = $1",
  [15]
);
const row = result.rows[0];

// Prisma
const product = await prisma.product.findUnique({
  where: { id: 15 }
});
```

Что важно

- Оба способа читают товар с id = 15 из PostgreSQL.
- В первом варианте SQL пишется вручную, во втором используется API модели.

Что такое модель Prisma?

- Модель — описание сущности: полей, типов, ключей и связей.
- Модель Product соответствует товарам, модель Category — категориям.
- Модель описывается в prisma/schema.prisma, но сами записи хранятся в PostgreSQL.
- Модель не создаёт отдельное хранилище данных в Node.js.

Главная идея

Таблица хранит данные; модель описывает их представление для Prisma.

Таблица products и модель Product

PostgreSQL products	Prisma Product	Смысл
id INTEGER	id Int	Идентификатор
name VARCHAR(150)	name String	Название
price INTEGER	price Int	Цена
category_id INTEGER	categoryId Int	Внешний ключ
in_stock BOOLEAN	inStock Boolean	Наличие

Две модели: Product и Category

Product

- id, name, price, categoryId, inStock
- Соответствует таблице products
- Каждый товар связан с одной категорией

Category

- id, name
- Соответствует таблице categories
- Одна категория связана со многими товарами

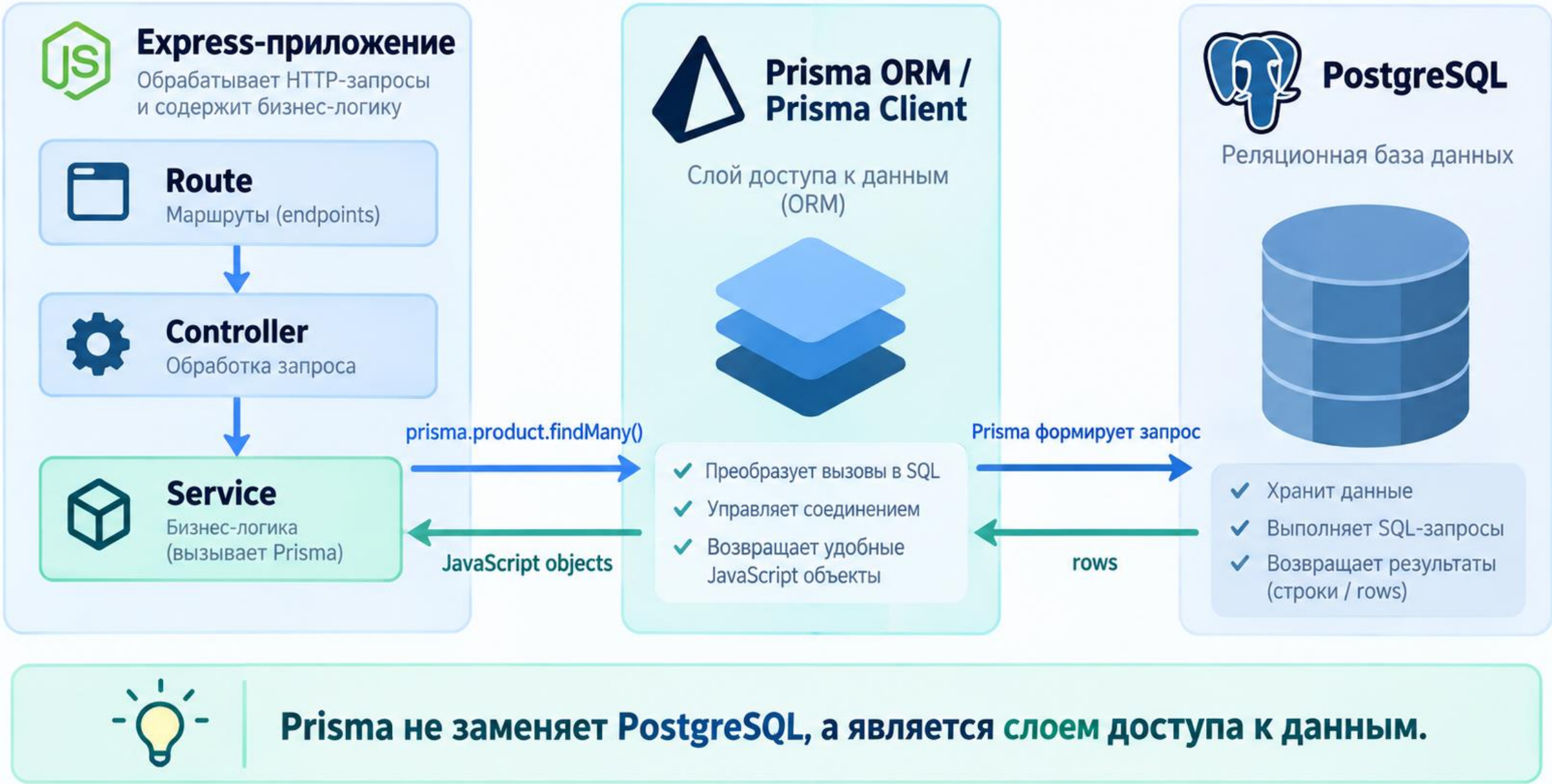
Что ORM не делает автоматически

- Не заменяет изучение SQL.
- Не устраняет ошибки проектирования реляционной схемы.
- Не отменяет индексы, ограничения и транзакции.
- Не гарантирует эффективность любого сформированного запроса.
- Для сложных операций иногда требуется SQL.

Архитектура Prisma

Как Prisma связывает JavaScript и существующую базу

Место Prisma ORM между приложением и PostgreSQL



Prisma не заменяет PostgreSQL, а является слоем доступа к данным.

Основные части Prisma ORM

Компонент	Назначение
Prisma Schema	Описание моделей, полей, ключей и связей.
Prisma Client	Сгенерированные методы запросов к БД.
Prisma Migrate	Изменения структуры БД с помощью миграций.
Prisma Studio	Графический просмотр и редактирование данных.

Три основных компонента

- PostgreSQL хранит реальные строки, индексы и ограничения.
- Prisma Schema описывает модели, атрибуты и связи.
- Prisma Client предоставляет методы JavaScript для чтения и изменения данных.
- Prisma не заменяет PostgreSQL: запросы выполняет сервер базы.

Главная идея

Prisma Schema — описание, Prisma Client — интерфейс запросов, PostgreSQL — данные.

Откуда появляются модели в нашем проекте?

- База shop_db создана в предыдущих лабораторных.
- Таблицы products и categories уже существуют и связаны через FOREIGN KEY.
- Команда prisma db pull анализирует существующую структуру.
- Prisma записывает найденные модели в schema.prisma, не удаляя данные.

Как работает introspection (db pull)

1	**Шаг 1**	Prisma подключается к shop_db по DATABASE_URL.
2	**Шаг 2**	Читает таблицы и столбцы PostgreSQL.
3	**Шаг 3**	Анализирует PRIMARY KEY, FOREIGN KEY и ограничения.
4	**Шаг 4**	Обновляет описания моделей в schema.prisma.
5	**Шаг 5**	После проверки моделей разработчик запускает prisma generate.

Когда используют db pull и Prisma Migrate

- db pull: база уже существует, модели получают из неё.
- Prisma Migrate: структура базы изменяется на основе Prisma Schema и миграций.
- В этой лекции Prisma подключается к существующим таблицам.
- Без необходимости не выполняем reset или команды, удаляющие данные.

Prisma Schema и связи

Описание двух таблиц после изменений в БД

Что такое Prisma Schema

- Файл prisma/schema.prisma описывает модели, связи и настройки генератора.
- model задаёт сущность, поля соответствуют столбцам БД.
- Атрибуты Prisma используют символ @.
- После db pull имена моделей и полей могут соответствовать исходным именам таблиц; при необходимости применяют @map и @@map.

Простая модель Product

```prisma
model Product {
  id         Int      @id @default(autoincrement())
  name       String   @db.VarChar(150)
  price      Int
  inStock    Boolean  @default(true) @map("in_stock")
  categoryId Int      @map("category_id")

  category   Category @relation(fields: [categoryId], references: [id])

  @@map("products")
}
```

- Модель Product описывает существующую таблицу products без создания новой таблицы.
- categoryId обязателен, так как products.category_id имеет NOT NULL.
- Текстовый столбец products.category уже удалён.

Модель Category

PRISMA

```
model Category {
  id       Int       @id
  name     String    @db.VarChar(100)
  products Product[]

  @@map("categories")
}
```

Что важно

- Категория хранится в таблице categories; поле называется name.
- products Product[] — обратная сторона связи 1:N.
- Не добавляйте @unique или автогенерацию id, если их нет в реальной БД.

Связь Product → Category

PRISMA

```
categoryId Int @map("category_id")
category   Category @relation(fields: [categoryId], references: [id])
```

- categoryId — реальный NOT NULL внешний ключ.
- category — поле отношения в Prisma; значение хранится в связанной таблице.
- Атрибут @relation записываем одной строкой для корректного разбора Prisma Schema.

PostgreSQL и Prisma Schema

PostgreSQL	Prisma
INTEGER	Int
VARCHAR	String @db.VarChar(n)
BOOLEAN	Boolean
TIMESTAMPTZ	DateTime @db.Timestamptz(...)
PRIMARY KEY	@id
UNIQUE	@unique
DEFAULT	@default(...)
FOREIGN KEY	@relation(...) и поля связи

Таблица PostgreSQL превращается в модель Prisma

Одна и та же структура данных — в базе и в коде приложения

PostgreSQL

База данных

products

#	Колонка	Тип (пример)
1	id	SERIAL
2	name	TEXT
3	price	NUMERIC(10, 2)
4	category_id	INTEGER
5	in_stock	BOOLEAN

Таблица PostgreSQL

Имена в snake_case, как обычно в базе данных

PRIMARY KEY → @id

category_id → @map("category_id")

products → @@map("products")

Prisma

Код приложения

```
model Product {
  id         Int      @id
  name       String
  price      Decimal
  categoryId Int      @map("category_id")
  inStock    Boolean

  @@map("products")
}
```

Модель Prisma

Имена в camelCase для удобной работы в коде

Имена в snake_case в базе данных могут стать camelCase в модели Prisma

@map и @@map

```prisma
categoryId Int     @map("category_id")
inStock    Boolean @map("in_stock")

@@map("products")
```

Что важно

- @map задаёт физическое имя столбца.
- @@map задаёт физическое имя таблицы.
- Так можно использовать camelCase и имена моделей в единственном числе.

Связи моделей и названия в коде

- categoryId — числовое поле модели Product, соответствующее category_id.
- category — relation-поле, позволяющее получать объект Category.
- products Product[] — обратная сторона связи в Category.
- При чтении можно использовать include: { category: true }.
- При создании товара нужно указывать существующий categoryId.

Установка и настройка Prisma 7

Пошаговая работа в существующем shop-api

Какую версию используем в курсе

- Используем ветку Prisma ORM 7 и ESM.
- Рабочий Express-проект остаётся на JavaScript.
- Стандартный генератор prisma-client создаёт TypeScript-код.
- Для его выполнения во время разработки используем tsx.

Главная идея

JavaScript-приложение не требуется переписывать на TypeScript.

Наша исходная ситуация

- Проект shop-api уже содержит Express-приложение и слой Service.
- PostgreSQL Server запущен, база shop_db содержит существующие таблицы.
- Product Service работает через pg и pool.query().
- Prisma подключаем к той же базе без её пересоздания.
- Приложение сохраняет файлы .js и архитектуру Route → Controller → Service.

Устанавливаем Prisma ORM 7

BASH

```
npm install -D prisma@7 typescript tsx @types/node @types/pg
npm install @prisma/client@7 @prisma/adapter-pg pg dotenv
```

- -D добавляет инструменты разработки в devDependencies.
- typescript и @types/* поддерживают TypeScript-инструменты Prisma.
- tsx исполняет JavaScript-проект, который импортирует сгенерированные .ts-модули.
- @prisma/adapter-pg подключает Prisma Client к PostgreSQL через pg.

Назначение пакетов

Пакет	Назначение
prisma	CLI: init, db pull, generate, studio
@prisma/client	Компоненты Prisma Client
@prisma/adapter-pg	Драйверный адаптер PostgreSQL
pg	Драйвер PostgreSQL
dotenv	Загрузка DATABASE_URL из .env
typescript	Инструменты TypeScript для инфраструктуры клиента
tsx	Запуск JS вместе с импортируемыми TS-файлами
@types/node, @types/pg	Описания типов для TS-инструментов

Инициализируем Prisma

BASH

```
npx prisma@7 init --datasource-provider postgresql --output ../generated/prisma
```

- init создаёт prisma/schema.prisma и конфигурацию Prisma CLI.
- PostgreSQL выбран как источник данных.
- Путь ../generated/prisma отсчитывается от файла prisma/schema.prisma.
- Сгенерированные файлы клиента появятся после prisma generate.

Что появляется в проекте

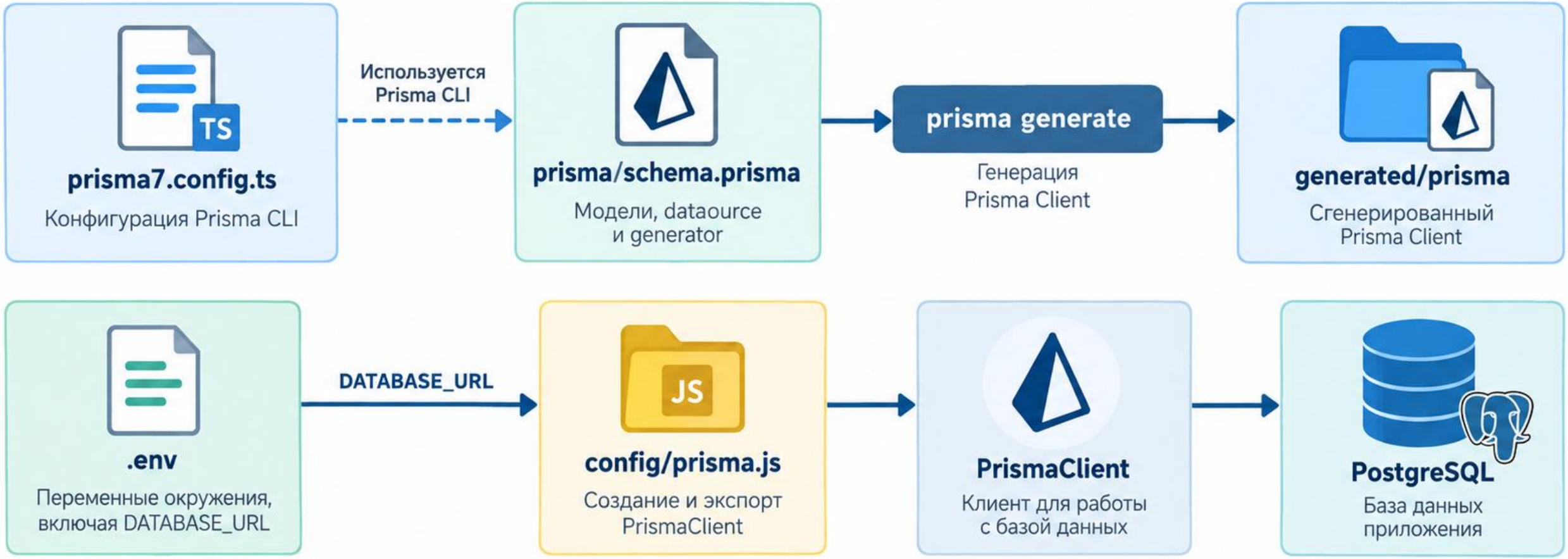
```text
shop-api/
├── prisma/
│   └── schema.prisma
├── generated/
│   └── prisma/          # после generate
├── prisma.config.ts  # имя файла см. после init
├── config/
│   ├── db.js
│   └── prisma.js      # создадим позднее
├── controllers/
├── routes/
├── services/
├── .env
├── app.js
└── package.json
```

Что важно

- Файл конфигурации CLI находится в корне проекта и создаётся командой init.
- config/prisma.js мы создаём сами; его использует Node.js приложение.
- До успешной проверки Prisma старый config/db.js и SQL-код не трогаем.

Файлы Prisma и их назначение

Как файлы связаны между собой и как Prisma работает в проекте



Ключевые идеи:

1. **.env** — хранит переменные окружения (например, DATABASE_URL).
2. **prisma7.config.ts** — используется Prisma CLI / конфигурацией схемы.
3. **schema.prisma** — содержит модели, datasource и generator, используется для генерации клиента.
4. **prisma generate** — создает Prisma Client в папке generated/prisma.
5. **config/prisma.js** — создает и экспортирует PrismaClient, который подключается к PostgreSQL.

Больше, чем код — понимание

Строка подключения DATABASE_URL

```text
DATABASE_URL="postgresql://shop_user:college123@localhost:5432/shop_db?schema=public"
```

- Строка задаёт протокол, пользователя, пароль, узел, порт и базу.
- Пример пароля допустим только в изолированной учебной среде.
- Файл .env не следует добавлять в Git.

Конфигурация Prisma CLI

TYPESCRIPT

```typescript
// prisma.config.ts
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: { path: "prisma/migrations" },
  datasource: { url: process.env.DATABASE_URL }
});
```

Что важно

- Редактируйте файл конфигурации Prisma, фактически созданный командой init, в корне shop-api.
- Конфигурация CLI нужна для db pull, generate и других команд Prisma.
- Её не требуется импортировать в JavaScript-файл app.js.

Генератор клиента для JavaScript-проекта

```
PRISMA
// блок generator в prisma/schema.prisma
generator client {
  provider             = "prisma-client"
  output               = "../generated/prisma"
  runtime              = "nodejs"
  moduleFormat         = "esm"
  importFileExtension = "ts"
}
```

Что важно

- Prisma ORM 7 с генератором prisma-client создаёт TypeScript-файлы.
- moduleFormat = "esm" соответствует нашему проекту ES Modules.
- importFileExtension = "ts" гарантирует правильное разрешение внутренних импортов при запуске через tsx.
- Сохраните этот блок после выполнения db pull.

Подключаем Prisma к существующей базе

1	**Шаг 1**	Установить пакеты Prisma CLI, Client, адаптер и инструменты tsx.
2	**Шаг 2**	Выполнить prisma init и проверить генератор в schema.prisma.
3	**Шаг 3**	Настроить DATABASE_URL и prisma.config.ts.
4	**Шаг 4**	Выполнить prisma db pull для чтения схемы существующей БД.
5	**Шаг 5**	Выполнить prisma generate и запустить тест через tsx.

Анализируем существующую базу

BASH

```
npx prisma@7 db pull
```

Что важно

- Команда считывает таблицы, поля, ограничения и связи.
- Модели создаются или обновляются в prisma/schema.prisma.
- db pull не удаляет данные существующей базы.
- После команды проверьте сохранность настроек generator.

Проверка результата db pull

- Откройте prisma/schema.prisma и найдите модели таблиц.
- В именах моделей после introspection возможны отличия от примеров.
- В дальнейшем будем использовать имена Product и Category; при необходимости скорректируйте модели и @@map.
- Проверьте, что существующие данные в shop_db сохранились.

Генерируем Prisma Client

```bash
npx prisma@7 generate
```

Что важно

- Генератор анализирует schema.prisma и записывает код в generated/prisma.
- В числе файлов появляется client.ts и другие TypeScript-модули.
- После db pull или изменения generator команду повторяют.

Слайд №34

schema.prisma → prisma generate → Prisma Client

На основе моделей из schema.prisma генерируется Prisma Client с типизированным API для работы с базой данных.

Prisma
Build faster
with less boilerplate

1 Модели в schema.prisma

Мы описываем структуру данных, поля и связи.

prisma/schema.prisma

```
model Product {
  id          Int      @id @default(autoincrement())
  name        String   @db.VarChar(150)
  price       Int
  categoryId  Int      @map("category_id")
  inStock     Boolean  @default(true) @map("in_stock")
  category    Category @relation(fields: [categoryId],
                       references: [id])
  orderItems  OrderItem[]

  @@map("products")
}

model Category {
  id          Int      @id @default(autoincrement())
  name        String   @unique @db.VarChar(100)
  products    Product[]

  @@map("categories")
}

model Order {
  id          Int      @id @default(autoincrement())
  userId      Int      @map("user_id")
  status      String   @db.VarChar(30)
  createdAt   DateTime @default(now()) @map("created_at")
  items       OrderItem[]
  user        User     @relation(fields: [userId], references: [id])
  @@map("orders")
}
```

Модели в schema.prisma — это описание существующей или будущей структуры базы данных.

2 Запуск генерации

Prisma CLI читает schema.prisma и создаёт клиент.

```
PS C:\project> npx prisma generate

Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma

✔ Generated Prisma Client (v7.x.x) to ./generated/prisma
  in 320ms
```



На основе моделей из **schema_prisma** генерируется типизированный API.

3 Сгенерированный Prisma Client

Готовый API для работы с моделями из schema.prisma.

generated/prisma/client

```
import { PrismaClient } from '../generated/prisma/client'

const prisma = new PrismaClient()

// Работа с моделью Product
const products = await prisma.product.findMany()
const product = await prisma.product.findUnique({
  where: { id: 1 }
})
const newProduct = await prisma.product.create({
  data: { name: 'Keyboard', price: 15000, categoryId: 1 }
})
const updatedProduct = await prisma.product.update({
  where: { id: 1 },
  data: { price: 12000 }
})
await prisma.product.delete({
  where: { id: 1 }
})
```

API формируется на основе моделей

- Для каждой модели создаётся свой namespace (prisma.product, prisma.category, prisma.order).
- Доступны типизированные методы: findMany, findUnique, create, update, delete и другие.
- Клиент учитывает поля, типы и связи, указанные в schema.prisma.
- После изменения моделей нужно запустить prisma generate повторно.

Модели → Генерация → Готовый API

Вы описываете структуру данных, а Prisma создаёт удобный и типизированный клиент.

Код Prisma Client генерируется автоматически

Файлы в папке generated/prisma изменять вручную не нужно.

Создаём один экземпляр PrismaClient

```javascript
// config/prisma.js
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client.ts";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
});

export const prisma = new PrismaClient({ adapter });
```

- Prisma ORM 7 использует драйверный адаптер PostgreSQL.
- Конфигурация клиента хранится в одном модуле.
- Импорт client.ts работает при запуске через tsx.
- Не создавайте экземпляр PrismaClient в каждом обработчике HTTP.

Как запускать Express с Prisma 7

```JSON
"type": "module",
"scripts": {
  "start": "tsx app.js",
  "dev": "tsx watch app.js",
  "prisma:generate": "prisma generate"
}
```

Что важно

- Это фрагмент package.json, а не полный файл: сохраняйте существующие зависимости и поля.
- npm start запускает приложение с поддержкой .ts-модулей Prisma Client.
- npm run dev автоматически перезапускает приложение при изменении файлов.
- Обычный node app.js для этой конфигурации не используем.

Проверка подключения и первого чтения

Проверяем Prisma без изменения Express-сервиса

Проверяем клиент до подключения Express

```javascript
// test-prisma.js
import { prisma } from "./config/prisma.js";

try {
  const products = await prisma.product.findMany();
  console.log(products);
} catch (error) {
  console.error(error);
} finally {
  await prisma.$disconnect();
}

// Запуск: npx tsx test-prisma.js
```

Что важно

- В консоли ожидается массив товаров из shop_db.
- Если имя модели после db pull отличается, сначала проверьте schema.prisma.
- Тестирование выполняется отдельно от HTTP-маршрутов.

Чтение категории с помощью Prisma Client

```javascript
// test-categories.js
import { prisma } from "./config/prisma.js";

try {
  const categories = await prisma.category.findMany();
  console.log(categories);
} finally {
  await prisma.$disconnect();
}

// Запуск: npx tsx test-categories.js
```

Что важно

- Проверяем доступность второй модели через prisma.category.
- Достаточно увидеть массив объектов с полями id и name.
- Пока не добавляем новый HTTP-маршрут.

Пример результата тестового чтения Product

```JSON
[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 15000,
    "inStock": true,
    "categoryId": 1
  }
]
```

Что важно

- Пример одного элемента массива; реальные ID и значения зависят от базы.
- Без include связь category не загружается.
- Подробный разбор findMany() и связанных запросов — в лекции № 6.

Prisma Studio

```bash
npx prisma@7 studio
```

Что важно

- Команда открывает интерфейс просмотра моделей через браузер.
- Studio работает с подключённой shop_db.
- Это инструмент разработчика, а не публичный frontend.

Мини-практика: Prisma подключена к shop_db

Выполнить

- Проверить таблицы products и categories в pgAdmin.
- Выполнить db pull; проверить модели и связи.
- Сгенерировать Prisma Client.
- Запустить test-prisma.js и test-categories.js.

Проверить

- schema.prisma соответствует базе.
- id, name, categoryId, inStock отображаются корректно.
- В консоли выводятся массивы из обеих таблиц.
- Существующие Express-маршруты и Product Service не изменены.

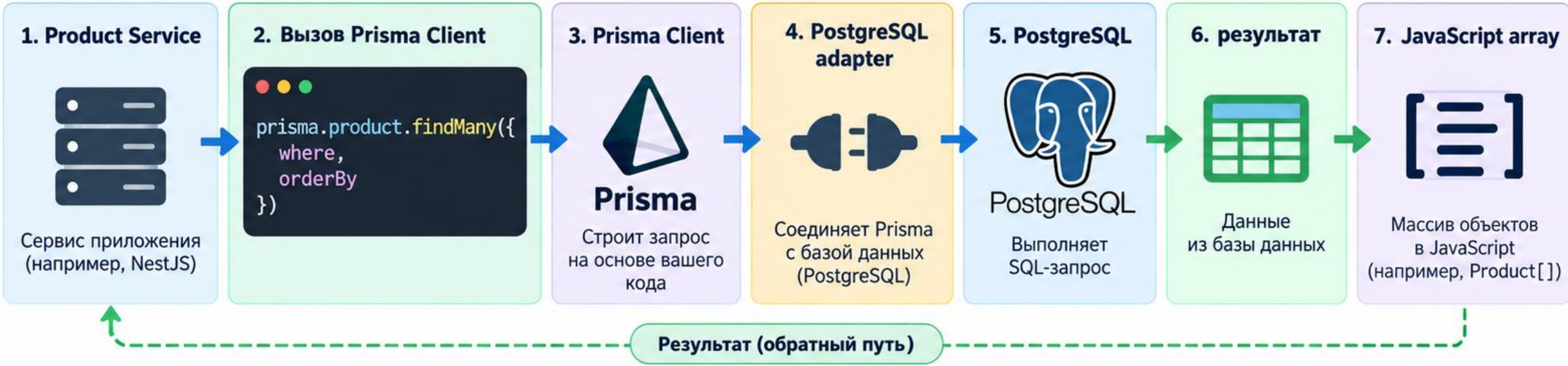
Контрольные вопросы

1. Что такое ORM?
2. Чем модель Prisma отличается от таблицы PostgreSQL?
3. Назовите назначение prisma/schema.prisma.
4. Что делает prisma db pull с существующей БД?
5. Для чего нужны @map и @@map?
6. Что означает обязательный categoryId Int в Product?
7. Какую роль играет @relation?
8. Зачем запускать prisma generate?
9. Чем prisma.config.ts отличается от config/prisma.js?
10. Почему тестовые скрипты выполняются через tsx?

Итоги занятия

1. PostgreSQL продолжает хранить реальные данные; Prisma не заменяет базу.
2. Prisma Schema описывает поля и связи существующих таблиц.
3. db pull получает структуру shop_db, generate создаёт Prisma Client.
4. config/prisma.js предоставляет один клиент JavaScript-приложению.
5. Тесты чтения подтверждают подключение к Product и Category.
6. Создание, изменение, удаление и полноценный REST CRUD относятся к лекции № 6.

Путь запроса prisma.product.findMany() до PostgreSQL



Эквивалентный SQL (упрощённо)

```sql
SELECT *
FROM "Product"
WHERE ...
ORDER BY ...;
```



Разработчик не пишет SQL вручную!

Prisma формирует SQL автоматически на основе вызова `prisma.product.findMany()`.

Кто за что отвечает?

Product Service
Вызывает методы ORM (Prisma Client).

Prisma Client
Строит запрос на основе вашего кода (prisma.product.findMany()).

PostgreSQL adapter
Соединяет Prisma с базой данных (PostgreSQL).

PostgreSQL
Выполняет SQL-запрос и возвращает данные.

JavaScript array
Результат приходит в виде массива объектов в JavaScript.